EVM 온체인 검증 시스템 상세 분석

개요

Proof of SQL의 EVM 온체인 검증 시스템은 Rust로 생성된 영지식 증명을 Ethereum 블록체인에서 직접 검증할 수 있도록 하는 Solidity 구현입니다. 이 시스템은 HyperKZG commitment 스킴과 sumcheck 프로토콜을 EVM 환경에서 효율적으로 실행하며, 가스 최적화를 통해 실용적인 온체인 검증을 가능하게 합니다.

아키텍처 개요

시스템 구성 요소

```
Rust Prover → Proof Generation → Solidity Verifier → EVM Execution

↓ ↓ ↓ ↓

SQL Query Binary Proof Gas-Optimized On-chain

Processing Serialization Verification Validation
```

핵심 컴포넌트

1. **Verifier.sol**: 메인 검증 라이브러리
2. **HyperKZGVerifier.sol**: HyperKZG commitment 검증
3. **Sumcheck.sol**: Sumcheck 프로토콜 구현
4. **ECPrecompiles**: 타원곡선 연산 최적화
5. **MathUtil**: 모듈러 산술 연산

메인 검증 시스템

Verifier.sol 구조

주요 함수 시그니처:

```
library Verifier {
function __verify(
bytes calldata __result,
bytes calldata __plan,
uint256[] memory __placeholderParameters,
bytes calldata __proof,
uint256[] memory __tableLengths,
uint256[] memory __commitments
) public view;
}
```

라인별 분석:

- `bytes calldata __result` : 쿼리 실행 결과 데이터
- `bytes calldata __plan` : SQL 실행 계획의 직렬화된 형태
- `uint256[] memory __placeholderParameters` : 쿼리 매개변수 값들
- `bytes calldata __proof` : Rust에서 생성된 영지식 증명
- `uint256[] memory __tableLengths` : 각 테이블의 행 수
- `uint256[] memory __commitments` : 테이블 데이터의 commitment 값들

검증 과정의 주요 단계

```
function __verify(...) public view {
assembly {
// 1. 첫 번째 라운드 메시지 읽기
proof_ptr, range_length, num_challenges :=
read_first_round_message(proof_ptr_init, transcript_ptr, builder_ptr)
// 2. 최종 라운드 메시지 읽기
proof_ptr, num_constraints :=
read_final_round_message(proof_ptr, transcript_ptr, builder_ptr)
// 3. Sumcheck 증명 검증
proof_ptr, evaluation_point_ptr :=
read_and_verify_sumcheck_proof(proof_ptr, transcript_ptr, builder_ptr, num_vars)
// 4. PCS 평가값 읽기 및 검증
proof_ptr := read_pcs_evaluations(proof_ptr, transcript_ptr, builder_ptr)
verify_pcs_evaluations(proof_ptr, commitments_ptr, transcript_ptr, builder_ptr,
}
}
```

단계별 분석:

1. **첫 번째 라운드**: Chi 평가값, Rho 평가값, 첫 번째 라운드 commitment들 처리
2. **최종 라운드**: 최종 commitment들과 비트 분포 정보 처리
3. **Sumcheck 검증**: 다항식 합 검증을 통한 증명 유효성 확인

4. **PCS 검증**: Polynomial Commitment Scheme 평가값 검증

HyperKZG 온체인 검증

HyperKZGVerifier.sol 구현

핵심 검증 함수:

```
function __verifyHyperKZG(
bytes calldata __proof,
uint256[1] memory __transcript,
uint256[2] memory __commitment,
uint256[] memory __x,
uint256 __y
) external view {
assembly {
// 1. Fiat-Shamir 트랜스크립트 실행
let r, q, d := run_transcript(proof_ptr, v_ptr(proof_ptr, ell), w_ptr(proof_ptr,
// 2. 이변수 평가 계산
let b := bivariate_evaluation(v_ptr(proof_ptr, ell), q, d, ell)
// 3. v 배열 일관성 검사
check_v_consistency(v_ptr(proof_ptr, ell), r, x, y)
// 4. L 그룹 원소 계산
compute_gl_msm(proof_ptr, sub(ell, 1), w_ptr(proof_ptr, ell), commitment_ptr, r,
// 5. R 그룹 원소 계산
```

```
univariate_group_evaluation(w_ptr(proof_ptr, ell), d, 3, add(scratch, WORDX6_SIZ

// 6. 페어링 방정식 검증

if iszero(ec_pairing_x2(scratch)) { err(ERR_HYPER_KZG_PAIRING_CHECK_FAILED) }

}

}
```

수학적 검증 공식

이변수 평가 계산:

```
b = Σ(i=0 to ℓ-1) Σ(j=0 to 2) q^i * d^j * v[i][j]
```

v 배열 일관성 검사:

```
0 = r * (2*v[i+1][2] + (x[i] - 1) * (v[i][1] + v[i][0])) + x[i] * (v[i][1] - v[i
```

페어링 검증:

```
e(L, -H) + e(R, τH) = 0
```

가스 최적화 기법

타원곡선 연산 최적화

ECPrecompiles.sol:

```
/// @dev Address of the ECADD precompile.
uint256 constant ECADD_ADDRESS = 0x06;
/// @dev Address of the ECMUL precompile.
uint256 constant ECMUL_ADDRESS = 0x07;
/// @dev Address of the ECPAIRING precompile.
uint256 constant ECPAIRING_ADDRESS = 0x08;


/// @dev Gas cost for the ECADD precompile.
uint256 constant ECADD_GAS = 150;
/// @dev Gas cost for the ECMUL precompile.
uint256 constant ECMUL_GAS = 6000;
/// @dev Gas cost for the ECPAIRING precompile with two pairings.
uint256 constant ECPAIRINGX2_GAS = 45000 + 2 * 34000;
```

라인별 분석:

- `ECADD_ADDRESS = 0x06` : EVM의 타원곡선 덧셈 프리컴파일 주소
- `ECMUL_ADDRESS = 0x07` : EVM의 타원곡선 스칼라 곱셈 프리컴파일 주소
- `ECPAIRING_ADDRESS = 0x08` : EVM의 페어링 연산 프리컴파일 주소
- 각 연산의 가스 비용이 미리 정의되어 효율적인 가스 추정 가능

메모리 레이아웃 최적화

Constants.sol:

```
/// @dev Size of a word in bytes: 32.
```

```
uint256 constant WORD_SIZE = 0x20;

/// @dev Size of two words in bytes.

uint256 constant WORDX2_SIZE = 0x20 * 2;

/// @dev Size of three words in bytes.

uint256 constant WORDX3_SIZE = 0x20 * 3;

// ... 최대 WORDX12_SIZE까지 정의



/// @dev Position of the free memory pointer in the context of the EVM memory.

uint256 constant FREE_PTR = 0x40;
```

최적화 특징:

1. **사전 정의된 크기**: 반복적인 곱셈 연산 제거
2. **메모리 정렬**: 32바이트 경계에 맞춘 효율적인 메모리 접근
3. **배치 처리**: 여러 연산을 한 번에 처리하여 가스 절약

Yul 어셈블리 최적화

인라인 어셈블리 사용:

```
assembly {

// 직접적인 메모리 조작

mstore(add(scratch, WORDX2_SIZE), G2_NEG_GEN_X_IMAG)

mstore(add(scratch, WORDX3_SIZE), G2_NEG_GEN_X_REAL)

// 효율적인 프리컴파일 호출

if iszero(staticcall(ECPAIRINGX2_GAS, ECPAIRING_ADDRESS, scratch, WORDX12_SIZE,
```

```
err(ERR_HYPER_KZG_PAIRING_CHECK_FAILED)

}

}
```

최적화 효과:

- **직접 메모리 접근**: Solidity 오버헤드 제거
- **인라인 연산**: 함수 호출 비용 절약
- **배치 메모리 할당**: 메모리 할당/해제 횟수 최소화

BN254 곡선 파라미터

곡선 상수 정의

Constants.sol:

```
/// @dev The modulus of the bn254 scalar field.

uint256 constant MODULUS = 0x30644e72_e131a029_b85045b6_8181585d_2833e848_79b970



/// @dev The X coordinate of the G1 generator point.

uint256 constant G1_GEN_X = 1;

/// @dev The Y coordinate of the G1 generator point.

uint256 constant G1_GEN_Y = 2;



/// @dev The G2 generator point's x-coordinate real component.
```

```
uint256 constant G2_GEN_X_REAL = 0x1800deef121f1e76426a00665e5c4479674322d4f75ed

/// @dev The G2 generator point's x-coordinate imaginary component.

uint256 constant G2_GEN_X_IMAG = 0x198e9393920d483a7260bfb731fb5d25f1aa493335a9e
```

라인별 분석:

- `MODULUS` : BN254 곡선의 스칼라 필드 모듈러스 (254비트 소수)
- `G1_GEN_X/Y` : G1 그룹의 생성원 좌표 (타원곡선 위의 점)
- `G2_GEN_*` : G2 그룹의 생성원 좌표 (확장 필드 위의 점, 실수부/허수부)

신뢰할 수 있는 설정 파라미터

```
uint256 constant VK_TAU_HX_REAL = 0x285b1f14edd7e6632340a37dfae9005ff762edcfecfe

uint256 constant VK_TAU_HX_IMAG = 0x17cc93077f56f654da727c1def86010339c2b4131094

uint256 constant VK_TAU_HY_REAL = 0x2bad9a374aec49d329ec66e8f530f68509313450580c

uint256 constant VK_TAU_HY_IMAG = 0x219edfceee1723de674f5b2f6fdb69d9e32dd53b1584
```

파라미터 설명:

- `VK_TAU_H*` : HyperKZG 검증을 위한 신뢰할 수 있는 설정 파라미터
- 이 값들은 ceremony를 통해 생성되며 검증자의 공개 키 역할
- G2 그룹의 원소로 페어링 연산에서 사용

데이터 타입 및 인코딩

컬럼 타입 변형

Constants.sol:

```
/// @dev Boolean variant constant for column types

uint32 constant DATA_TYPE_BOOLEAN_VARIANT = 0;

/// @dev TinyInt variant constant for column types

uint32 constant DATA_TYPE_TINYINT_VARIANT = 2;

/// @dev SmallInt variant constant for column types

uint32 constant DATA_TYPE_SMALLINT_VARIANT = 3;

/// @dev Int variant constant for column types

uint32 constant DATA_TYPE_INT_VARIANT = 4;

/// @dev BigInt variant constant for column types

uint32 constant DATA_TYPE_BIGINT_VARIANT = 5;

/// @dev Varchar variant constant for column types

uint32 constant DATA_TYPE_VARCHAR_VARIANT = 7;

/// @dev Decimal75 variant constant for column types

uint32 constant DATA_TYPE_DECIMAL75_VARIANT = 8;
```

인코딩 최적화:

- 각 데이터 타입에 고유한 변형 번호 할당
- Rust와 Solidity 간 일관된 타입 매핑
- 효율적인 직렬화/역직렬화 지원

바이트 크기 상수

```
/// @dev Size of boolean in bytes. In bincode encoding, booleans are one byte.
```

```
uint256 constant BOOLEAN_SIZE = 0x01;

/// @dev Number of bits needed to pad boolean to 256 bits

uint256 constant BOOLEAN_PADDING_BITS = 0xF8;



/// @dev Size of uint64 in bytes

uint256 constant UINT64_SIZE = 0x08;

/// @dev Number of bits needed to pad uint64 to 256 bits

uint256 constant UINT64_PADDING_BITS = 0xC0;
```

패딩 최적화:

- 각 데이터 타입의 정확한 바이트 크기 정의
- 256비트 워드로 패딩하기 위한 비트 수 사전 계산
- 비트 시프트 연산을 통한 효율적인 데이터 추출

오류 처리 시스템

오류 코드 정의

Errors.sol:

```
library Errors {

error InvalidECAddInputs();

error InvalidECMulInputs();

error InvalidECPairingInputs();

error HyperKZGEmptyPoint();
```

```
error HyperKZGProofSizeMismatch();

error HyperKZGPairingCheckFailed();

}
```

오류 처리 특징:

1. **타입 안전성**: 각 오류에 고유한 선택자 제공
2. **가스 효율성**: 커스텀 오류를 통한 가스 절약
3. **디버깅 지원**: 명확한 오류 메시지로 문제 진단 용이

입력 검증

```
function verify_hyperkzg(proof_ptr, transcript_ptr, commitment_ptr, x, y) {

let ell := mload(x)

// 빈 평가 점 검사

if iszero(ell) { err(ERR_HYPER_KZG_EMPTY_POINT) }

// 증명 크기 일치 검사

let com_len := shr(UINT64_PADDING_BITS, calldataload(proof_ptr))

if sub(com_len, sub(ell, 1)) { err(ERR_HYPER_KZG_PROOF_SIZE_MISMATCH) }

}
```

검증 단계:

1. **크기 검증**: 배열 길이와 증명 크기 일치 확인
2. **범위 검증**: 모든 값이 유효한 필드 원소인지 확인
3. **구조 검증**: 증명 데이터의 내부 구조 일관성 검사

성능 벤치마크

가스 사용량 분석

| 연산 | 가스 비용 | 설명 |

|------|----------|------|

| ECADD | 150 | 타원곡선 점 덧셈 |

| ECMUL | 6,000 | 타원곡선 스칼라 곱셈 |

| ECPAIRING (2개) | 113,000 | 이중 페어링 연산 |

| 전체 검증 | ~500,000 | 일반적인 쿼리 검증 |

최적화 효과

메모리 사용량 최적화:

- **사전 할당**: 필요한 메모리를 미리 계산하여 할당
- **재사용**: 스크래치 메모리 공간의 효율적 재활용
- **정렬**: 32바이트 경계 정렬을 통한 접근 속도 향상

연산 최적화:

- **배치 처리**: 여러 타원곡선 연산을 한 번에 처리
- **프리컴파일 활용**: EVM 네이티브 암호학 연산 최대 활용
- **인라인 어셈블리**: Solidity 오버헤드 제거

Rust-Solidity 일관성 보장

직렬화 형식 통일

Rust 측 (bincode):

```rust
#[derive(Serialize, Deserialize)]
pub struct QueryProof {
pub first_round_message: FirstRoundMessage,
pub final_round_message: FinalRoundMessage,
pub sumcheck_proof: SumcheckProof,
pub pcs_evaluations: PcsEvaluations,
}
```

Solidity 측 파싱:

```solidity
function read_first_round_message(proof_ptr_init, transcript_ptr, builder_ptr) -
range_length := shr(UINT64_PADDING_BITS, calldataload(proof_ptr))
proof_ptr := add(proof_ptr, UINT64_SIZE)
num_challenges := shr(UINT64_PADDING_BITS, calldataload(proof_ptr))
proof_ptr := add(proof_ptr, UINT64_SIZE)
// ... 추가 필드 파싱
}
```

타입 안전성 보장

컴파일 타임 검증:

1. **상수 일치**: Rust와 Solidity의 상수 값 자동 동기화
2. **구조체 레이아웃**: 메모리 레이아웃 일관성 검증
3. **테스트 커버리지**: 크로스 플랫폼 테스트를 통한 호환성 확인

보안 고려사항

입력 검증

```
// 모듈러스 범위 검증
uint256 constant MODULUS_MASK = 0x1FFFFFFF_FFFFFFFF_FFFFFFFF_FFFFFFFF_FFFFFFFF_F

function validate_field_element(value) {
if gt(value, MODULUS_MINUS_ONE) { err(ERR_INVALID_FIELD_ELEMENT) }
}
```

재진입 공격 방지

```
function __verify(...) public view {
// view 함수로 선언하여 상태 변경 방지
// staticcall만 사용하여 외부 호출 제한
}
```

메모리 안전성

```
// 메모리 오버플로우 방지
let scratch := mload(FREE_PTR)
mstore(FREE_PTR, add(scratch, required_size))
```

결론

Proof of SQL의 EVM 온체인 검증 시스템은 복잡한 영지식 증명을 Ethereum 블록체인에서 효율적으로 검증할 수 있도록 설계된 고도로 최적화된 시스템입니다.

핵심 성과:

- **가스 효율성**: ~500,000 가스로 완전한 SQL 쿼리 검증
- **보안성**: 엄격한 입력 검증과 타입 안전성 보장
- **호환성**: Rust 구현과 완벽한 일관성 유지
- **확장성**: 다양한 SQL 연산과 데이터 타입 지원

이러한 온체인 검증 시스템을 통해 Proof of SQL은 탈중앙화된 환경에서 신뢰할 수 있는 SQL 쿼리 실행을 보장하며, 블록체인 기반 데이터 분석의 새로운 가능성을 열어줍니다.